The method travel_back resets time to its original state.

It's sometimes useful to put the following line in a setup method:

```
travel_to(Date.today)
```

and then the other half in the teardown method:

```
travel_back
```

This ensures that the current time doesn't change for the duration of each test. Again, with certain timing-related issues, that consistency eliminates a possible source of intermittent test failures or just plain confusion.

The argument to travel_to is a Date, Time, DateTime, or anything that accepts the message to_time. The argument to travel is an integer number of seconds that is added to the current time.

Both methods also take blocks such that the fake time is good only for the duration of the block:

```
it "reports based on start date" do
  travel_to(Date.parse("2015-02-10")) do
    actual = Project.find_started_in_last(6.months)
    expect(actual.size).to eq(3)
  end
end
```

The time-travel methods can be in your setup or in an individual test. You can use travel to change the time in the middle of a test to speed up an ongoing process:

```
it "knows if the project is over" do
  p = Project.new(:start_date => Date.today,
      :end_date = Date.today + 8.weeks)
  expect(p).not_to be_complete
  travel(10.weeks)
  expect(p).to be_complete
end
```

These methods let you keep explicit dates in your test data without causing problems later. The only downside is that if you have many tests setting time to different days, it can get somewhat confusing in the aggregate. It's easier if you use the same start date consistently. (On a solo project, you might use your birthday, for instance, but that's probably overly cute for a team project.) A more minor problem is that the line at the end of your test runs that indicates how long the test suite took may be hopelessly messed up because of the continued tweaking of Time.now.